fixtures/tier_route/ — T self-validation fixtures

Fixture convention for `constitution/scripts/fastcycle/context/tier_route.py` (plan.md T-E06; tasks.md T109/T118; research.md DEC-28; SC-005, FR-023). Task T109 (`tasks.md`) writes ONLY this RED test + these fixtures; the tool itself is T118, a later, separate task (Producer != Verifier, §11.4.240 — this file's author never implements `tier_route.py`).

No contract file exists for this tool. `specs/004-fast-dev-cycles/contracts/common-conventions.md` 's own "Tool map" section lists `$FC/context/tier_route.py (T-E06)` under "Plan tools that no contract in this directory covers ... Their interface, output and RED fixtures are fixed by the plan task text until a contract is written." Unlike T104/T108 (which borrowed CLI verb names and field shapes from a real contract file and only DEFINED the under-specified corners), this file defines the ENTIRE interface — verb name included — from plan.md T-E06's + tasks.md T109/T118's own wording alone, flagged explicitly as binding-if-adopted, never claimed as something a contract already fixed.

The routing rule this fixture set encodes (from plan.md T-E + research.md DEC- , restated precisely, never invented)

1. **Reviews are hard-excluded.** A review-class task is REFUSED by this router outright — never assigned any tier, cheap or otherwise. Review tier/effort assignment belongs exclusively to the ALREADY-LANDED `$FC/review/review_record.py` 's own `RB-004` refusal (its real `DESIGNATED_TIER = "opus"` / `DESIGNATED_EFFORT = "xhigh"` constants, checked live by this

suite's test script, never assumed) and to `contracts/common-conventions.md` 's "Designated review tier" term ("never lowered or routed (T-C11, T-E06)"). `tier_route.py` must never re-decide this.

2. **A task class WITH a deterministic verifier** starts at the lightest tier declared to fit the window (`min_fitting_tier` — a GIVEN classification input in this fixture set, exactly as `governance_subset` 's `classification_inputs` are given rather than computed by that suite's fixtures; the window-fit determination itself is T-E02/T114's concern, not this tool's) and escalates EXACTLY ONE ladder rung on each verifier failure, never more, never fewer, until a `pass` or the ladder is exhausted (exhaustion behaviour is UNCONFIRMED by the plan text and explicitly OUT OF SCOPE for this fixture set — see "Explicit scope exclusion" below).
3. **A task class WITHOUT a deterministic verifier** (and not a review) is NEVER routed below the constitution's own default working tier (`sonnet` , §11.4.231(A)) — it is not refused, not treated as an error, and not silently defaulted to the cheapest tier either; it PASSES at the safe default with zero escalation attempts. This is the "negative control" that proves the safety net is a real PASS, not a disguised refusal.

Tier ladder (fixed by this fixture set, ascending — closed, three rungs)

`["haiku", "sonnet", "opus"]` — mirrors this project's constitution §11.4.231(A)'s `sonnet < opus` default working ladder EXTENDED with `haiku` as the lightest rung, per this project's own §11.4.231(D.1) re-admission clause and plan.md T-E02's stated goal ("a haiku-tier dispatch of a representative item now fits its window"). `sonnet` is the DEFAULT working tier (§11.4.231(A)); `opus` / `xhigh` is the SEPARATE, pinned review substrate (§11.4.209/§11.4.211/§11.4.231(E)) — never a rung this router assigns to a review, only the rung `review_record.py` independently pins outright.

Layout

Four self-contained fixtures, one JSON file each, `schema: "tier-route-fixture/v1"` :

Fixture	Kind	What it proves
`tr_golden_escalation.json`	golden	T109's own named golden scenario: a single verifier failure at the lightest fitting tier escalates exactly one rung, then passes
`tr_golden_double_escalation.json`	golden (bonus, defence-in-depth)	escalation walks ONE RUNG PER FAILURE even across two consecutive failures — never a "give up and jump to the top" short-cut
`tr_bad_review_routed_cheap.json`	golden-bad	T109's own named golden-bad scenario: a review-class task is refused out-right, never assigned any tier
`tr_negctrl_no_verifier.json`	negative-control	T109's own named negat-ive-control scenario: a non-review class with no verifier routes to the safe default, PASSING rather than er-roring or refusing

Wire format (UNCONFIRMED — no contract exists — DEFINED here,[118]

binding-if-adopted on T)

Each fixture's `expected_route` block is the interim output contract: `{refused: bool, refusal_reason: str|null, route_sequence: [tier,...], escalations: int, final_tier: str|null}`. `refused=true` corresponds to C-001 exit code 1 ("A finding: refusal ..."); `refused=false` corresponds to exit code 0. This suite's `derive_route` (embedded in `test_tier_routing_red.sh`, never imported by nor shared with the not-yet-existing `context/tier_route.py`, section 11.4.245) is the DERIVED oracle every fixture's own `expected_route` is independently re-verified against before that value is trusted for anything downstream (mirroring `test_governance_subset_red.sh`'s `derive_selection` / `test_evidence_ref_red.sh`'s `derive_consume` precedent exactly).

The (speculative, this-file-defined) forward-compatible CLI this suite's "real-tool invocation" block exercises once T118 lands: `context/tier_route.py route --fixture <fixture.json> --out <route.json>` — a `route` verb chosen for consistency with the `select` / `verify` / `consume` verb-naming convention established by `governance_subset.py` / `evidence_ref.py`; T118 is free to choose a different verb, at which point this block (and only this block — every fixture and the derived-oracle checks above it are interface-agnostic) needs adjusting at T118 review time, exactly per `common-conventions.md`'s "Their interface ... fixed by the plan task text until a contract is written" convention applied to a tool with NO contract at all.

Explicit scope exclusion (section . . — honestly out of scope, not silently absorbed)

Two aspects of T118's own task text are deliberately NOT exercised by this fixture set, mirroring T108's own "explicit scope exclusion" precedent for `create` / `reverify`:

- **Ladder exhaustion** (every rung's verifier fails, including `opus`). Neither plan.md T-E06 nor tasks.md T109/T118 specifies what happens next (a permanent refusal? a hand-off to a human? an operator-blocked state per §11.4.21?) — inventing an answer here would be a section-11.4.6 guess. `derive_route` below raises on this input shape rather than silently returning a fabricated verdict; T118's own implementation decision, once made, is this fixture set's to extend with a fifth fixture at review time.
- **"Calibrate on recorded outcomes"** (T118's own closing clause). This requires a recorded-outcomes data store this spec has not yet defined anywhere (no contract, no data-model entity, no plan task that produces one ahead of T118) — out of scope for a RED test that must derive its expectations from data that already exists or is fixed by task text alone.